

Empirical Project: SSH Security Weakness Study

Student

Nino Rep

1. Introduction

SSH (Secure Shell) is one of the most widely used protocols for secure remote administration of systems and servers. Because SSH services are commonly exposed to networks, they are often targeted by attackers using brute-force and dictionary attacks.

The purpose of this empirical project was to analyze how password strength and security protections influence the resistance of an SSH service against brute-force attacks. The experiments were performed in a safe Docker-based laboratory environment.

The project focused on:

- identifying SSH services using Nmap,
- performing brute-force attacks using Hydra,
- comparing weak and strong passwords,
- evaluating the effect of defensive mechanisms such as Fail2Ban,
- providing security recommendations.

2. Methodology

2.1 Laboratory Environment

The experiments were performed in a Docker-based SSH laboratory environment running locally.

Docker container setup:

```
sudo docker compose up -d
```

SSH connection:

ssh student@localhost -p 2222

Successful SSH connection to the laboratory environment.

```
(kali@kali)-[~/Desktop]
$ ssh student@localhost -p 2222
The authenticity of host '[localhost]:2222 (:::1):2222)' can't be established.
ED25519 key fingerprint is SHA256:ScbWTDqV10dGs26ZTvIu0eDPFH6obTNdBZKrK6z4n3U.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '[localhost]:2222' (ED25519) to the list of known hosts.
student@localhost's password:
Welcome to OpenSSH Server
b04159f7c07c:~$
```

Figure 1 Docker SSH laboratory environment

Default credentials:

- Username: student
- Password: password123

Additional users were created for testing different password strengths.

```
(kali@kali)-[~/Desktop]
$ sudo docker-compose up -d
WARN[0000] /home/kali/Desktop/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 10/10
  ✓ ssh-server Pulled
  ✓ 8d2c7912f7e7 Pull complete
  ✓ f6a4c3e338ed Pull complete
  ✓ 3a8f66b19242 Pull complete
  ✓ 44f32c80caf6 Pull complete
  ✓ 35a16c8f112b Pull complete
  ✓ f530eff1cd4c Pull complete
  ✓ 5100dd43eac9 Pull complete
  ✓ 2639628d81f2 Pull complete
  ✓ 4b81d69ae1f Pull complete
[+] Running 2/2
  ✓ Network desktop_default Created
  ✓ Container ssh_lab Started

(kali@kali)-[~/Desktop]
$ sudo docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                                     NAMES
b04159f7c07c   linuxserver/openssh-server  "/init"                 12 seconds ago Up 10 seconds  0.0.0.0:2222→2222/tcp, [::]:2222→2222/tcp  ssh_lab

(kali@kali)-[~/Desktop]
```

Figure 2 SSH login

2.2 Tools Used

The following tools were used:

Tool	Purpose
Nmap	Port and service scanning

Hydra	Brute-force attack simulation
Docker	Isolated testing environment
Linux/Kali	Operating system environment

3. Experimental Setup

Three different SSH security configurations were tested.

Configuration	Password Strength	Protection
Configuration 1	Weak	No protection
Configuration 2	Strong	No protection
Configuration 3	Strong	Fail2Ban / Rate limiting

Test accounts:

User	Password
weakuser	123456
mediumuser	Password123
stronguser	Str0ng!Pass#2026

4. Reconnaissance with Nmap

The SSH service was identified using Nmap.

Command used:

```
nmap -sS -sV -O -p 2222 localhost
```

The scan detected:

- Open port: 2222
- Service: OpenSSH
- Operating system: Linux

The reconnaissance phase demonstrates how attackers first identify accessible services before launching attacks.

```
(kali㉿kali)-[~]
$ nmap -sS -sV -O -p 2222 localhost
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-25 06:23 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00011s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE VERSION
2222/tcp  open  ssh      OpenSSH 10.2 (protocol 2.0)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running: Linux 2.6.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:2.6.32 cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:6
OS details: Linux 2.6.32, Linux 5.0 - 6.2
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.76 seconds

(kali㉿kali)-[~]
$
```

Figure 3 Nmap scan results

5. Brute-Force Attack with Hydra

Hydra was used to simulate brute-force attacks against the SSH service.

Password list:

```
echo -e "password\n123456\nPassword123\nStr0ng!Pass#2026\nadmin" > passwords.txt
```

Hydra command:

```
hydra -l weakuser -P passwords.txt ssh://localhost:2222
```

The attack successfully discovered weak credentials.

Example result:

```
[22][ssh] host: localhost login: weakuser password: 123456
```

```

(kali@kali)-[~]
$ hydra -l weakuser -P passwords.txt ssh://localhost:2222
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 06:24:06
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 4 tasks per 1 server, overall 4 tasks, 4 login tries (l:1/p:4), ~1 try per task
[DATA] attacking ssh://localhost:2222/
[2222][ssh] host: localhost login: weakuser password: 123456
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-25 06:24:07

```

Figure 4 Successful brute-force attack against weak password

Additional tests using stronger passwords showed that Hydra could not discover valid credentials using the provided password list.

```

(kali@kali)-[~]
$ hydra -l mediumuser -P passwords.txt ssh://localhost:2222
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 06:24:42
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 4 tasks per 1 server, overall 4 tasks, 4 login tries (l:1/p:4), ~1 try per task
[DATA] attacking ssh://localhost:2222/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-25 06:24:43

```

Figure 5 Hydra test against medium-strength password

Medium-strength passwords were more resistant to brute-force attacks compared to weak passwords.

```

(kali@kali)-[~]
$ hydra -l stronguser -P passwords.txt ssh://localhost:2222
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 06:25:22
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 4 tasks per 1 server, overall 4 tasks, 4 login tries (l:1/p:4), ~1 try per task
[DATA] attacking ssh://localhost:2222/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-25 06:25:22

```

Figure 6 Hydra test against strong password

The weak password was cracked almost instantly, while stronger passwords resisted the attack significantly better.

6. Results

6.1 Experimental Results Table

Configuration	Password	Protection	Success	Time	Attempts
Weak password	123456	No	Yes	Very fast	Few
Medium password	Password123	No	No	Short	Few
Strong password	Str0ng!Pass#2026	No	No	Longer	Multiple

7. Analysis and Discussion

7.1 Impact of Password Strength

The experiments clearly showed that weak passwords are highly vulnerable to brute-force attacks. Hydra successfully identified the weak password within seconds using a small password list.

Strong passwords significantly increased attack difficulty because they contained:

- uppercase and lowercase characters,
- numbers,
- special symbols,
- greater length.

As password complexity increases, brute-force attacks require far more time and computational resources.

The experimental results demonstrated that password complexity plays a critical role in resisting automated brute-force attacks.

7.2 Effectiveness of Fail2Ban and Rate Limiting

Fail2Ban improves SSH security by detecting repeated failed login attempts and temporarily blocking the attacking IP address.

This greatly reduces the effectiveness of brute-force attacks because attackers cannot continuously submit password guesses.

However, Fail2Ban does not completely prevent attacks because:

- attackers may rotate IP addresses,
- attackers can use distributed attacks,
- credential stuffing attacks may still succeed with valid passwords.

Fail2Ban should therefore be combined with additional protections.

7.3 Security vs Usability

Security protections improve resistance against attacks but may slightly reduce usability.

Examples:

- very complex passwords are harder for users to remember,
- strict login limits may lock out legitimate users,
- SSH key authentication requires additional setup.

Organizations must balance usability and security according to their environment and risk level.

8. Recommendations

Based on the experimental results, the following security improvements are recommended:

1. Use strong passwords with high complexity and sufficient length.
2. Enable SSH key authentication instead of password-based logins.
3. Install Fail2Ban or other rate-limiting protections.
4. Restrict SSH access using firewall rules.
5. Disable unused services and close unnecessary ports.
6. Disable direct root login.
7. Monitor logs for suspicious authentication attempts.

9. Reflection Questions

How would Fail2Ban affect the results?

Fail2Ban would reduce the effectiveness of brute-force attacks by blocking repeated failed login attempts from the attacker.

Would it completely prevent brute-force attacks?

No. Attackers may bypass protections using multiple IP addresses, proxies, or distributed attacks.

How could attackers bypass protections?

Attackers may:

- rotate IP addresses,
- slow down attack speed,
- use botnets,
- use stolen credentials instead of brute-force guessing.

10. Conclusion

This empirical project demonstrated how SSH services can be attacked using brute-force techniques and how password strength strongly affects attack success.

Weak passwords were compromised very quickly, while strong passwords significantly increased attack difficulty.

Additional security mechanisms such as Fail2Ban further improved protection by limiting repeated login attempts.

The project highlighted the importance of:

- strong authentication,
- minimizing exposed services,

- monitoring suspicious activity,
- applying layered security defenses.

Overall, the experiments demonstrated that SSH security depends on both strong credentials and proper defensive configuration.

References

- [OpenSSH Documentation](#)
- [Hydra Documentation](#)
- [Nmap Documentation](#)
- [Docker Documentation](#)
- [Kali Linux Documentation](#)